

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automatics

Mihail Pancenco FAF-243

Report

Laboratory work No. 4

Operating Systems

The xv6 Shell from the Outside

Supervisors:
Dumitraşcu Marius, University assistant
Reiţman Patricia, University assistant
DISA, FCIM, UTM

Chişinău – 2026

Table of Contents

1 Introduction **3**

2 Tasks and results **3**

2.1 Environment 3

2.2 Part 1 - Sleep on the installed xv6 3

2.3 Part 2 - Bash versus the xv6 shell 4

2.4 Part 3 - A program that prints argv 7

2.5 Part 4 - Read user/sh.c 8

3 Conclusions **10**

A Submitted source code **11**

A.1 user/sleep.c 11

A.2 user/args.c 11

A.3 args_linux.c 11

1 Introduction

Compare Bash with the xv6 shell, inspect the argument vector delivered to a program, and connect observed behavior to the shell implementation.

The experiments follow the laboratory handout and were performed inside Ubuntu 26.04.1 LTS in VirtualBox. The figures show actual terminal screenshots taken during execution; complete transcripts and the submitted source files accompany the report.

2 Tasks and results

2.1 Environment

All experiments were performed on 23 October 2026 inside **Ubuntu 26.04.1 LTS (ARM64)** running in VirtualBox as user `misa` on `UbuntuVM`. The Linux kernel is `7.0.0-38-generic`. The installed xv6 checkout is `~/xv6-riscv`, commit `06aad25c735fd3159bdfae5680be4eab7b1668b2`. GCC and the RISC-V compiler are version 15.2.0; QEMU is version 10.2.1. xv6 is booted from this Ubuntu with `make qemu CPUS=1`.

The figures are screenshots of the Ubuntu terminal during execution. Complete command output is provided in `terminal-output.txt`; build logs are in `build-output.txt` for the xv6 labs. PIDs, inode numbers and free-space values below refer to these recorded runs.

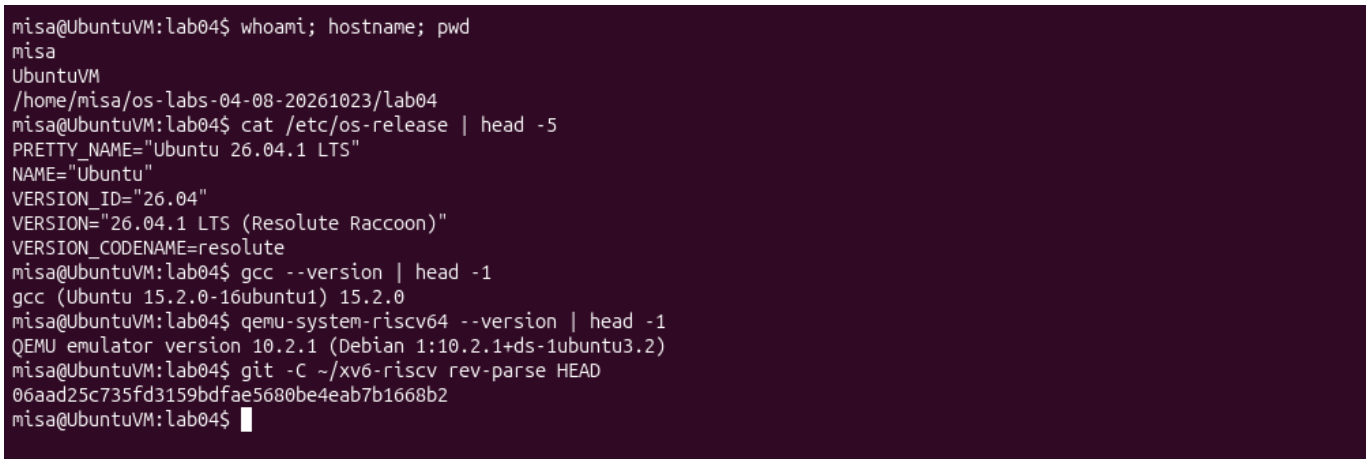


Figure 2.1 - Ubuntu environment, user, compiler, QEMU and installed xv6 revision.

2.2 Part 1 - Sleep on the installed xv6

`user/user.h` declares `pause(int)`, so `user/sleep.c` calls `pause(atoi(argv[1]))`. The command remains named `sleep`, and its argument is a number of timer ticks. Exactly one argument is required; otherwise it prints usage and exits with status 1. The Makefile has entries for both `sleep` and `args` in `UPROGS`.



Figure 2.2 - The pause declaration, UPROGS entries and successful filesystem-image build.

`sleep 20` returned to the shell after its pause; `sleep` without an argument printed the expected usage message. The same compiled utility is used in the subsequent xv6 experiments.

2.3 Part 2 - Bash versus the xv6 shell

The Bash comparison was performed in a new directory containing `README`. xv6 used a freshly built image with the lab utilities. The complete comparison is:

Command or feature	Bash	xv6
`echo hello > f ; cat < f`	Prints `hello`.	Prints `hello`; redirection and ordered lists work.
`ls | grep READ`	Prints `README`.	Prints the README row with type, inode and byte size.
`(echo a; echo b) | wc`	`2 2 4`	`2 2 4`
`echo one ; echo two`	Prints `one`, then `two`.	Same order.
`echo bg &`	Runs in the background and displays job/PID information.	Runs in the background without Bash's job table.
`ls *`	Expands the existing names `README` and `f`.	`ls: cannot open *`; the asterisk is literal.
`echo $HOME`	`/home/misa`	Literal `$HOME`.
`echo "a b"`	`a b`	`"a b"`, with the quotes retained.
`mkdir d ; cd d ; ls ..`	Enters `d` and lists its parent.	`exec cd failed`; the shell stays in the root directory.
Up, Tab, Ctrl+R	History, completion and reverse search work.	These interactive facilities are absent.

```
$ sleep 20
$ sleep
usage: sleep <ticks>
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$
```

Figure 2.3 - Redirection, pipes, lists, background execution and literal expansion syntax in xv6.

```
misa@UbuntuVM:compare$ echo hello > f ; cat < f
hello
misa@UbuntuVM:compare$ ls | grep READ
README
misa@UbuntuVM:compare$ (echo a; echo b) | wc
      2       2       4
misa@UbuntuVM:compare$ echo one ; echo two
one
two
misa@UbuntuVM:compare$ echo bg &
[1] 9707
misa@UbuntuVM:compare$ bg
wait
misa@UbuntuVM:compare$ ls *
README  f
misa@UbuntuVM:compare$ echo $HOME
/home/misa
misa@UbuntuVM:compare$ echo "a b"
a b
misa@UbuntuVM:compare$ mkdir d ; cd d ; ls ..
README  d  f
misa@UbuntuVM:d$
```

Figure 2.4 - The same shell comparison commands executed in Ubuntu Bash.

2.3.1 1. Features common to both shells and features added by Bash

Both support running executables, input/output redirection, pipes, semicolon-separated lists, background commands and parenthesized grouping. Bash additionally provides wildcard and variable expansion, quote processing, PATH search, richer built-ins, command history, completion and job management.

The xv6 LIST case explicitly forks the left command, calls `wait(0)`, and then runs the right command. Therefore `echo hello > f ; cat < f` is ordered and reads the newly written file; it is not a race between the two commands.

2.3.2 2. Why ls fails after cd, and how to run it

A standalone `cd d` changes the parent shell's working directory. xv6 does not search PATH; bare `ls` is interpreted as an executable relative to the current directory, while the program is stored in `/`. Use `/ls` or `../ls` from `d`.

```
$ cd d
$ ls
exec ls failed
$ /ls
.              1 27 32
..             1 1 1024
$ ../ls
.              1 27 32
..             1 1 1024
$ cd /
$
```

Figure 2.5 - Bare ls fails inside d, while absolute and parent-relative executable paths work.

In `mkdir d ; cd d ; ls ..`, the special parent-shell check does not see `cd` at the start of the line. The list parser treats it as a separate executable, which does not exist, producing `exec cd failed`. Even a real external cd could change only its own working directory, not that of the parent shell.

2.3.3 3. Which missing feature I would add

Executable search would matter most in daily use because changing directory otherwise breaks short program names. A simple fallback that tries `/command` after a failed relative exec would be the easiest useful first extension. Full Bash-compatible expansion and quoting require more parsing and expansion logic.

2.3.4 Interactive key checks

In Bash, Up recalled `echo history-check`, Tab completed `cat REA` to `cat README`, and Ctrl+R found the earlier command. The screenshot captures the active reverse-search match. In xv6, Up did not recall the previous command, Tab acted as whitespace rather than filename completion, and Ctrl+R did not initiate history search.

```
misa@UbuntuVM:compare$ echo history-check
history-check
misa@UbuntuVM:compare$ echo history-check
history-check
misa@UbuntuVM:compare$ cat README
(reverse-i-search)`history-check': echo history-check
```

Figure 2.6 - Bash history recall, filename completion and an active Ctrl+R reverse search.

```
$ echo history-check
histo failedk
$ args REA
argc = 2
argv[0] = [args]
argv[1] = [REA]
$
exec  failed
$
```

Figure 2.7 - The corresponding key checks in the minimal xv6 console.

2.4 Part 3 - A program that prints argv

`user/args.c` prints argc and every argument using xv6 headers and exits with `exit(0)`. `args_linux.c` uses `<stdio.h>` and returns 0. The Linux program was compiled with `gcc -Wall -Wextra -Werror -o args args_linux.c`. A separate Bash directory contained exactly `a.c` and `b.c`; the directory holding `args` was added to PATH.

```
$ args one   two  three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
$
```

Figure 2.8 - All four argument-vector experiments in the xv6 shell.

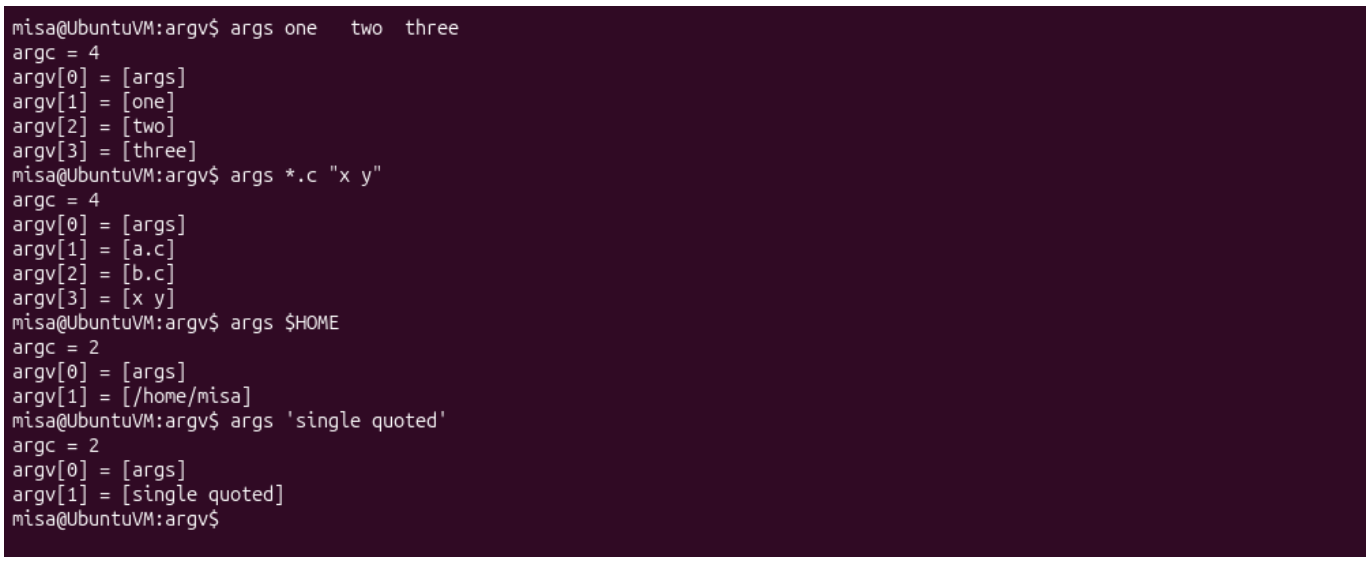


Figure 2.9 - The same argument-vector experiments in Ubuntu Bash.

2.4.1 1. Outputs for args *.c "x y"

Both have argc 4, including argv[0]. Bash passes three user arguments: `a.c`, `b.c`, and the single string `x y`. xv6 passes the literal `*.c` and the two strings `"x` and `y"`. Equal counts are coincidental: changing the number of matching C files changes Bash's argc.

2.4.2 2. Who expands the wildcard and removes quotes?

The shell performs both transformations before exec. The kernel accepts an executable pathname and an already constructed argument vector; the argument-printing program only displays what it receives. Bash also replaces `$HOME` with `/home/misa` and turns `'single quoted'` into one argument without the quote characters. xv6 passes `$HOME` literally and splits the quoted phrase into two strings retaining the quote characters.

2.4.3 3. What happened to repeated spaces?

For `args one two three`, both systems report argc 4 and arguments `one`, `two`, `three`. Runs of unquoted whitespace separate words and do not create empty arguments. Bash can preserve spaces using quotes; the xv6 tokenizer does not implement that quoting rule.

2.5 Part 4 - Read user/sh.c

The installed source has 499 lines. Its five command kinds and execution cases were inspected directly in Ubuntu:

```
misa@UbuntuVM:xv6-riscv$ wc -l user/sh.c
499 user/sh.c
misa@UbuntuVM:xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
misa@UbuntuVM:xv6-riscv$ grep -n "case " user/sh.c | head -5
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
misa@UbuntuVM:xv6-riscv$ sed -n "93,100p" user/sh.c
  case LIST:
    lcmd = (struct listcmd *)cmd;
    if (fork1() == 0)
      runcmd(lcmd->left);
    wait(0);
    runcmd(lcmd->right);
    break;

misa@UbuntuVM:xv6-riscv$ sed -n "166,170p" user/sh.c
    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
      // Chdir must be called by the parent, not the child.
      cmd[strlen(cmd) - 1] = 0; // chop \n
      if (chdir(cmd + 3) < 0)
        fprintf(2, "cannot cd %s\n", cmd + 3);
misa@UbuntuVM:xv6-riscv$
```

Figure 2.10 - Source inspection: command types, dispatch cases, LIST ordering and parent-shell cd handling.

2.5.1 1. Five command kinds and their syntax

Kind	Syntax	Execution
EXEC	A program name and arguments	`exec(argv[0], argv)` replaces the child image.
REDIR	<, >, »	Redirects a descriptor before executing the nested command.
PIPE	`|`	Connects two children through a pipe and waits for both.
LIST	`;`	Waits for the left command before running the right one.
BACK	`&`	Forks a command without waiting for its completion in that branch.

Parentheses group commands rather than defining a sixth kind. In this source, the execution cases begin at lines 75, 83, 93, 101 and 125 respectively.

2.5.2 2. Where cd is handled and why

In `main`, line 166 checks for a line beginning with `cd ` and line 169 calls `chdir(cmd + 3)` before the shell forks a command. The shell itself must make the call because the current working directory is a per-process property.

2.5.3 3. Why there is no wildcard or variable expansion

Literal searches for `*` and `$` were run on `user/sh.c`. Asterisks occur in pointer declarations and dereferences; `$` occurs in the prompt. No expansion routine resolves filenames or environment variables. The tokenizer splits words using whitespace and syntax symbols such as `<`, `>`, `|`, `&`, `;` and parentheses. This explains the literal asterisk, variable name and quote characters observed above.

3 Conclusions

The experiments distinguish shell syntax from kernel mechanisms. xv6 supplies process execution, lists, pipes and redirection, while Bash adds expansion, quoting, executable search and interactive facilities. Printing argv makes these differences explicit. Source inspection confirms ordered execution: the LIST branch waits before executing its right-hand command.

Bibliography

[1] Operating Systems, FCIM / FAF, Technical University of Moldova. Laboratory work No. 4: *The xv6 Shell from the Outside*. Course handout, 2026–2027.

[2] MIT PDOS. *xv6-riscv*, commit `06aad25c735fd3159bdfae5680be4eab7b1668b2`. Source checked on 23 October 2026. `https://github.com/mit-pdos/xv6-riscv/tree/06aad25c735fd3159bdfae5680be4eab7b1668b2`.

[3] Linux man-pages project. *fork(2), pipe(7), inode(7), path_resolution(7)*. `https://man7.org/linux/man-pages/`.

A Submitted source code

The following listings read the actual submitted source files, so the report and submission use the same implementation. Variant programs are included where relevant.

A.1 user/sleep.c

```c
#include "kernel/types.h"
#include "user/user.h"
int main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage: sleep <ticks>\n");
        exit(1);
    }
    pause(atoi(argv[1])); // system call into the kernel
    exit(0);
}
```

A.2 user/args.c

```c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
```

A.3 args_linux.c

```c
#include <stdio.h>

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  return 0;
}
```